Sistemas Distribuidos y Programación Paralela

Trabajo Práctico 3

Cloud Computing

Informe · Partes 0, 1 y 2

Grupo Cerberus

Salvador Baez (195157) · Mateo Nomico (168102) · Tomás Resnik (190168)

Universidad Nacional de Luján · Departamento de Ciencias Básicas
Cátedra: Dr. David Petrocelli · Comisión Mercedes · Segundo cuatrimestre de 2026
Lenguaje: Python 3.14 · Repositorio: github.com/chimera-sdypp/sdypp_tp3 · Rama: main
Noviembre de 2026

Contenido

1. Presentación
2. Parte 0 — Cluster local con k3s
3. Hit #0 — Patrones de mensajería con RabbitMQ
4. Hit #1 — Operador de Sobel distribuido
5. Marco teórico
6. Hit #2 — Sobel con offloading a la nube
7. Hit #3 — Sobel asincrónico en Kubernetes
8. Hit #4 — Observabilidad
9. Análisis de carga
10. Pruebas, CI/CD y seguridad
11. Herramientas de IA
12. Conclusiones
13. Referencias

1. Presentación

El TP3 lleva el operador de Sobel de una sola computadora a la nube en tres partes. La Parte 0 deja andando un cluster de Kubernetes local; la Parte 1 agrega mensajería con RabbitMQ y distribuye el cálculo con el patrón Master-Worker; la Parte 2 lo hace escalar de verdad: primero desbordando el cómputo hacia máquinas que se crean y se destruyen en la nube (Hit #2), y después con el sistema entero sobre Kubernetes, asincrónico y observable (Hits #3 y #4).

Cada hit está en su carpeta del repositorio, con un README que tiene las instrucciones de ejecución, el diagrama y las decisiones de diseño. Este informe resume cada uno, desarrolla el marco teórico que pide la Parte 2 y reúne las mediciones.

Hit	Carpeta	Qué hay	Health-check
Parte 0	hit1/README.md	Validación del cluster k3s	—
#0	TP3/queue/ex1 … ex5	Cuatro patrones de RabbitMQ y la comparación	—
#1	hit1/	Sobel centralizado, distribuido y tolerante a fallos	GET /health de cada worker
#2	hit2/	Master como servicio en k3s que crea VMs en GCP según la cola	GET /health del master
#3	hit3/	Sobel asincrónico en Kubernetes: backend, split, workers, joiner, reasignador, frontend	GET /health de cada servicio (por el ingress, el del backend)
#4	hit4/	Prometheus, Grafana, Alertmanager, Loki y Fluent Bit	Los de Prometheus y Grafana

Cada health-check devuelve un JSON con el estado del servicio y de lo que necesita, por ejemplo {"backend": "ok", "rabbitmq": "ok", "redis": "ok"}, y 503 si algo falta.

El Hit #3 corre en la nube en http://8.232.111.42 (/health devuelve el estado de todos los servicios): en el GKE tp3 del proyecto tp3distribuidos, que compartimos con otro grupo, en el namespace chimera-sobel y en un node pool propio (chimera). Nuestro GKE original (proyecto chimera-tp3, creado con hit3/infra) se borró el 04/11 para no consumir crédito y ese proyecto quedó sin facturación.

2. Parte 0 — Cluster local con k3s

El cluster local es k3s v1.36.4 sobre la PC de Tomás (CachyOS). k3s usa containerd y no Docker, así que las imágenes construidas con docker build se importan con docker save | sudo k3s ctr images import -. kubectl corre sin sudo con el kubeconfig copiado al usuario. Las cinco validaciones se hicieron el 07/10/2026:

Validación	Resultado
kubectl get nodes	El nodo en Ready
nginx-test con replicas: 2	2/2 pods; curl localhost:8080 → 200
Auto-healing	Se borró un pod y el Deployment creó otro
Importar una imagen	k3s ctr images import con la imagen del Hit #1 (la usa hit1/k8s/sobel.yaml)
Rolling update	Cambio de imagen a nginx:1.25-alpine

Los objetos que se usan en el resto del TP son los de la Parte 0: Pod, Deployment (réplicas y rolling update), Service (incluido el headless del Hit #1, que publica una IP por pod en el DNS), ConfigMap y Secret. La evidencia está en la sección "Prerrequisitos cumplidos" del README del Hit #1.

3. Hit #0 — Patrones de mensajería con RabbitMQ

Los cuatro ejemplos de TP3/queue usan RabbitMQ (AMQP 0-9-1) con pika y corren tanto con Docker Compose como en k3s, con sus manifiestos. El ex5 los compara y muestra dónde aparece cada patrón en el Hit #3.

Patrón	Cómo se arma en RabbitMQ	Qué se probó	Cuándo conviene
Ex1 · Cola punto a punto	Cola durable, mensajes persistentes (delivery_mode=2), ack manual y prefetch_count=1	10 tareas numeradas; con 2 consumidores se alternan 1-3-5-7-9 y 2-4-6-8-10 (round-robin)	Repartir trabajo entre workers iguales: cada mensaje lo procesa uno solo
Ex2 · Pub/Sub fanout	Exchange fanout; cada suscriptor ata su cola exclusiva y temporal	"Nuevo bloque minado": los 3 suscriptores reciben cada mensaje	Avisar un evento a varios interesados que el emisor no conoce
Ex3 · Dead Letter Queue	Cola con x-dead-letter-exchange; el consumidor hace nack(requeue=False)	Los mensajes con "error": true (3, 6 y 9) terminan en la DLQ con su cabecera x-death; otro consumidor los lee	Aislar mensajes que no se pueden procesar, sin frenar al resto
Ex4 · Retry con backoff	Colas de espera de 1, 2, 4 y 8 s con TTL que devuelven el mensaje a la cola principal	Falla al azar el 50 %; se loguea intento y espera; al 5.° fallo va a la DLQ	Fallas transitorias (un servicio que se recupera), sin reintentar sin límite

En el Hit #3 se combinan los cuatro: las tareas a los workers son colas punto a punto, los resultados salen por un fanout, lo que falla va a una DLQ y el reasignador lo reintenta con un límite. A partir de la corrección del 23/10, las credenciales de RabbitMQ de los ejemplos salen de un .env (con un .env.example en el repo) y, en Kubernetes, de un Secret: ya no hay usuario ni clave guest en el código ni en los manifiestos.

4. Hit #1 — Operador de Sobel distribuido

Etapa 1: centralizado.py aplica Sobel a una imagen (gradiente horizontal y vertical, magnitud recortada a 0–255). Etapa 2: el master parte la imagen en franjas horizontales, se las manda por HTTP a N workers (POST /sobel) y une los resultados: es el patrón Master-Worker o granja de trabajadores. Cada franja lleva una fila de borde extra arriba y abajo, que el master recorta al unir, así la salida es idéntica a la de la Etapa 1. Corre con Docker Compose (--scale worker=N) y en k3s, donde el master encuentra a los workers por un Service headless.

Etapa 3: tolerancia a fallos. Un worker se da por caído si corta la conexión, responde 5xx o algo que no es una imagen, o no contesta en SOBEL_TIMEOUT. Se lo descarta hasta el final del trabajo y su franja pasa al siguiente vivo; si no queda ninguno, el master vuelve a resolver el DNS antes de rendirse. Las franjas de un worker colgado vencen en paralelo, así que se pierde un timeout y no uno por franja. Con SIGTERM el worker termina lo que tiene en curso antes de salir, así que un apagado planificado no cuesta reasignaciones.

Evidencia de tests/tolerancia.sh (8 franjas, 2 workers sanos, 1 caído y 1 colgado, SOBEL_TIMEOUT=3):

```
franja 7/8: el worker 172.24.0.4:8080 no está disponible → worker descartado, la franja se reasigna
franja 7/8 (37 filas) → 172.24.0.2:8080 en 0.002 s (intento 2)
franja 4/8: el worker 172.24.0.5:8080 no respondió en 3 s → worker descartado, la franja se reasigna
4 reasignaciones; workers descartados: 172.24.0.4:8080, 172.24.0.5:8080
OK: salida idéntica a la centralizada con un worker caído y otro colgado
```

El mismo script mata con docker kill un worker sano a mitad del trabajo: sus franjas se reasignan y la salida también es idéntica. En k3s (14/10), con 12 franjas y los workers de k8s/fallas.yaml, hubo 4 reasignaciones, salida idéntica y el Deployment reinició el pod caído.

5. Marco teórico

5.1 Qué es cloud computing

La definición del NIST [NIST800-145] describe la nube como un modelo para acceder por red, cuando se lo necesita, a un conjunto compartido de recursos configurables que se aprovisionan y se liberan rápido, con un mínimo de gestión o de interacción con el proveedor. Pide cinco características esenciales; la tabla las cruza con lo que usa este TP:

Característica	Qué significa	Dónde aparece en el TP
Autoservicio bajo demanda	Se piden recursos sin hablar con nadie del proveedor	terraform apply crea el cluster y las VMs de workers; el mismo apply con cantidad = 0 borra las VMs
Acceso amplio por red	Los servicios se usan por red, con protocolos estándar	Ingress y health-checks públicos por HTTP; los workers se conectan por red al broker
Pool de recursos	El proveedor reparte hardware físico entre muchos clientes (multi-tenant)	Las VMs y los nodos de GKE corren sobre hosts compartidos; no se elige la máquina física
Elasticidad rápida	La capacidad crece y se achica con la demanda, y parece ilimitada	Workers creados por tarea en el Hit #2; réplicas y node pools que escalan en el Hit #3
Servicio medido	El uso se mide y se cobra por lo consumido	Facturación por segundo de VM; en nuestro lado, Prometheus mide colas, tareas y latencias

El NIST también distingue modelos de servicio (IaaS, PaaS, SaaS) y de despliegue (privado, comunitario, público e híbrido). En esos términos, el Hit #2 es un despliegue híbrido sobre IaaS: la parte local sigue en nuestras máquinas y las VMs de la nube son infraestructura cruda. El Hit #3 usa un Kubernetes administrado (GKE), un punto intermedio en el que el proveedor opera el plano de control y nosotros, los contenedores.

5.2 La economía de la nube según Armbrust et al.

El trabajo de Berkeley [ARMBRUST10] señala tres cosas que la nube trajo de nuevo: la ilusión de recursos infinitos disponibles al instante, la eliminación del compromiso por adelantado (no hay que comprar servidores antes de saber si hacen falta) y la posibilidad de pagar por uso en períodos cortos. De ahí sale la "asociatividad del costo": mil máquinas durante una hora cuestan lo mismo que una máquina durante mil horas, así que un trabajo paralelizable puede terminar mil veces antes por el mismo precio.

Es exactamente el caso de Sobel: los fragmentos de una imagen son independientes, así que más workers durante menos tiempo dan el mismo costo y una respuesta más rápida. El mismo trabajo enumera diez obstáculos; estos son los que pesan en nuestra arquitectura:

- **Cuello de botella en la transferencia de datos.** En el Hit #2 cada franja viaja de la red local a la nube y vuelve procesada. Para imágenes chicas, la red puede tardar más que el cálculo, y el tráfico de salida de la nube se paga.
- **Rendimiento impredecible.** Una VM comparte el host con otros clientes, así que la misma franja no tarda siempre lo mismo. Por eso el análisis de carga se repite y se reportan percentiles (p50, p95, p99), no promedios.
- **Escalar rápido.** Una VM tarda del orden de un minuto en arrancar y bajar la imagen, frente a segundos para un pod nuevo en un nodo que ya existe. Desbordar a la nube conviene para trabajos que duran bastante más que ese minuto.

- **Dependencia del proveedor (lock-in).** Terraform, los contenedores y Kubernetes la reducen, porque el mismo manifiesto corre en k3s y en GKE. Lo que queda atado al proveedor es la identidad de las máquinas (Workload Identity, cuentas de servicio).
- **Confidencialidad.** Las imágenes de los usuarios pasan por máquinas que no controlamos. Se mitiga con registry privado, secretos fuera del repo y acceso a los servicios internos restringido por red.

5.3 Offloading y cloud bursting

Hacer offloading [KUM10] es mandar cómputo a otra máquina más capaz cuando conviene: el costo de transferir los datos tiene que ser menor que lo que se gana en tiempo de cálculo. El cloud bursting [CBURST] es el caso particular en que la nube se usa como extensión de la capacidad propia: la carga normal se atiende en las máquinas locales, ya pagadas, y sólo cuando la demanda las satura el excedente "se desborda" hacia recursos alquilados por el tiempo justo. Así se paga el costo fijo de la base y el costo variable sólo en los picos. El Hit #2 implementa este patrón (sección 6).

5.4 El teorema CAP: dónde se paran RabbitMQ y Redis

Brewer planteó en 2000 [BREWER00] que un sistema distribuido no puede garantizar a la vez consistencia (todos leen el último valor escrito), disponibilidad (todo pedido a un nodo vivo recibe respuesta) y tolerancia a particiones (seguir funcionando aunque se corte la red entre nodos). Gilbert y Lynch lo demostraron en 2002 [GILBERT02]. Como en una red real las particiones pasan, la elección práctica es qué resignar mientras dura una: consistencia o disponibilidad. El mismo Brewer aclaró después [BREWER12] que fuera de una partición no hay que resignar nada.

RabbitMQ elige consistencia (CP). Nuestras colas son quorum queues [RMQ-QQ], que replican cada cola entre los nodos con el algoritmo de consenso Raft: un mensaje se confirma al productor recién cuando lo guardó la mayoría. Si la red se parte, sólo el lado con mayoría sigue aceptando y entregando; del lado de la minoría la cola deja de estar disponible. RabbitMQ prefiere rechazar un mensaje antes que confirmarlo y después perderlo. (Las colas espejadas clásicas, que podían divergir, se eliminaron en RabbitMQ 4.)

Redis prioriza disponibilidad y latencia. La replicación de Redis es asincrónica [REDIS-REPL]: el primario confirma una escritura antes de que la tengan las réplicas. Si el primario se cae o queda aislado y se promueve una réplica, las escrituras confirmadas que no llegaron a replicarse se pierden; la documentación de Redis Cluster aclara que no garantiza consistencia fuerte [REDIS-CLUSTER]. Es una base pensada para responder rápido, no para consensuar cada escritura.

Por qué importa para nuestra arquitectura:

- **Lo que no se puede perder viaja por RabbitMQ.** Cada tarea es un mensaje persistente en una cola quorum y el backend publica con confirmación del broker. Si RabbitMQ no puede confirmar, el backend contesta 503 en lugar de aceptar un job que después se perdería.
- **Las decisiones que no se pueden repetir se toman sobre un solo Redis.** Que el joiner una cada archivo una sola vez depende de operaciones atómicas (SADD, WATCH/MULTI) sobre el primario. Con una réplica promovida tras perder escrituras, dos joiners podrían creer que les toca unir el mismo archivo. Por eso Redis corre como una sola instancia con persistencia en disco (AOF), y no como un cluster con failover.
- **Repetir trabajo no rompe nada.** La entrega de RabbitMQ es al menos una vez: si un worker muere antes del ack, el fragmento se procesa de nuevo. Como cada paso escribe en Redis la misma clave con el mismo resultado, repetirlo es inofensivo (idempotente). Así los dos sistemas se complementan: RabbitMQ no pierde tareas y Redis tolera que se repitan.

- **El costo es la disponibilidad.** Con un solo Redis y un RabbitMQ que prefiere consistencia, una caída de cualquiera de los dos detiene la aceptación de jobs (503 con Retry-After) hasta que vuelve; los servicios reconectan con backoff y no se pierde lo ya aceptado.

6. Hit #2 — Sobel con offloading a la nube

El Hit #2 reutiliza el Sobel del Hit #1, que ya reparte franjas entre workers HTTP y reasigna las de un worker caído. Lo que agrega es que parte de esos workers no existe hasta que hace falta: Terraform los crea en la nube, atienden el trabajo y se destruyen. Es cloud bursting. Tuvo dos versiones: la primera (tag v1.0.0, entregada el 23/10) desbordaba una sola imagen a N VMs; la final, después de la corrección del 23/10, es un servicio que recibe pedidos todo el tiempo y crea o borra VMs según el trabajo que se acumula.

6.1 Versión final: el master como servicio que escala por la cola

El master del Hit #1 corre en el k3s de la PC como un servicio (localhost:30080) con cuatro partes en un solo proceso: la API (POST /pedidos responde 202 con un id; GET /pedidos/<id> da el estado y GET …/resultado el PNG), un consumidor de la cola sobel.peticiones de RabbitMQ, el escalador y una limpieza del disco. A la cola va sólo el id del pedido; la imagen queda en el disco del master (un PVC) y el estado en Redis. El ack se hace recién después de guardar el resultado, así que si el master se cae a mitad de un pedido, el pedido vuelve a la cola. Cada imagen se reparte en franjas entre 2 workers locales y las VMs que haya en ese momento.

- **Escalado por el largo de la cola.** Cada 5 s el escalador pide una VM cada 3 pedidos en espera, hasta 2. La cola es el trabajo pendiente real; la CPU no sirve, porque el master casi no la usa. Una VM se mantiene hasta que la cola baja de su umbral durante 30 s (como la ventana de estabilización del HPA), y antes de borrarla se la saca de la lista de workers y se espera el pedido en curso.
- **Terraform desde el pod, sin claves.** El escalador hace terraform apply -var cantidad=N sobre el mismo módulo y el mismo estado remoto que el CI. El pod cambia el token de su ServiceAccount de k3s por credenciales temporales de GCP (Workload Identity Federation con un pool para el k3s local), y las VMs bajan la imagen con su propia cuenta de servicio, que sólo puede leer el registry.
- **Ciclo de cada VM.** Provisioning (apply), bootstrap y deploy (Container-Optimized OS ya trae Docker; el startup-script sólo baja y arranca el worker), join (cuando responde /health entra en los pedidos siguientes) y teardown (apply con menos VMs, o con 0 cuando el master recibe SIGTERM).
- **Observabilidad.** El master expone /metrics (cola, VMs pedidas y listas, llegadas, atendidos y tiempos) con su ServiceMonitor, y un dashboard de Grafana muestra la cola contra los umbrales y los logs del master desde Loki.

Demo del 29/10: 80 fotos de 100 MB, 2 workers locales limitados a 450m (para que rindan como una VM e2-standard-2) y VMs en southamerica-east1-a. El cliente manda pedidos a un ritmo que se cambia en vivo. Todas las salidas fueron idénticas a la centralizada y al final quedaron 0 VMs.

Ritmo (pedidos/min)	Qué pasó
3,5	Los 2 workers locales alcanzan (3,7/min, 16,3 s por pedido): la cola queda en 0–1 y no se crea ninguna VM
4,7	A los ~2,5 min la cola llega a 3 y se crea una VM; con 3 workers (4,6–4,85/min) la cola queda en 2–3
5,9	A los ~3,5 min la cola llega a 6 y se crea la segunda; con 4 workers (5,8–6/min) queda en 4–6
5,9 → 4,7 → 3,5	Cada VM se borra ~1 min después de bajar el ritmo

6.2 Primera versión (v1.0.0): desborde de una imagen

La primera versión usaba el master del Hit #1 tal cual: acepta una lista de workers (--workers host:puerto,...), así que los locales y los de la nube trabajaban juntos sin cambiar su código. desbordar.sh creaba N VMs, procesaba una imagen y las borraba. Sus mediciones siguen siendo las que muestran cuándo conviene la nube.

Ciclo de vida de un worker en la nube

Etapa	Cómo
Provisioning	terraform apply crea N VMs con la cantidad pedida como variable
Bootstrap	El startup-script (el user_data de GCP, arranque.sh.tftpl) instala Docker en el arranque de la VM
Deploy	La VM baja la imagen del worker del registry con la identidad de su cuenta de servicio, sin claves
Join	El worker escucha en el 8080; terraform output da las IPs y el master las suma a su lista
Teardown	Al terminar la imagen, desbordar.sh vuelve a hacer terraform apply con cantidad = 0 y Terraform borra las VMs. Corre en un trap, así que pasa aunque algo falle. La cuenta de servicio y el firewall quedan: no cuestan nada y el próximo apply no tiene que recrearlos

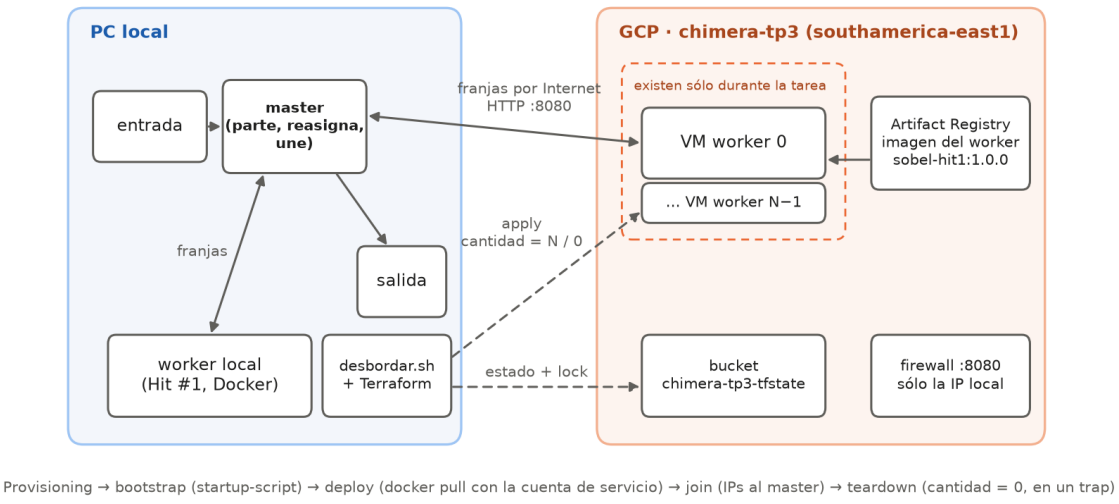


Figura 6.1 — Arquitectura híbrida del Hit #2: el master y un worker en la PC local, N workers efímeros en GCP.

Terraform

hit2/infra tiene los cuatro archivos separados (provider.tf, variables.tf, main.tf y outputs.tf). La cantidad de VMs es una variable: en 0 no existe ninguna, y el mismo apply sirve para crearlas y para borrarlas. Lo esencial:

```
# provider.tf: el estado vive en un bucket de GCS con lock; nunca en el repositorio
backend "gcs" { prefix = "hit2" }   # terraform init -backend-config="bucket=chimera-tp3-tfstate"

# main.tf: una VM por unidad de cantidad (0 por defecto)
resource "google_compute_instance" "worker" {
  count        = var.cantidad
  machine_type = var.tipo_maquina    # e2-standard-2
  metadata     = { startup-script = templatefile("arranque.sh.tftpl", { imagen = var.imagen, ... }) }
  service_account { email = google_service_account.worker.email }   # sólo lee el registry, sin claves
}

# desbordar.sh: el teardown corre siempre al salir, aunque algo falle
trap '"${TF[@]}" apply -auto-approve -input=false -var cantidad=0 >/dev/null' EXIT
```

Lo que se crea una sola vez (las APIs, el Artifact Registry y la identidad del CI) está en infra/base y se aplica a mano, porque incluye la cuenta con la que el CI aplica el resto. El pipeline terraform.yml valida en cada cambio, corre plan en cada PR y apply al mergear a main, autenticado por OIDC con Workload Identity: cada job recibe credenciales que vencen en una hora y no hay claves de GCP en GitHub. En main, hit2/infra queda con cantidad = 0, así que el apply del CI no deja VMs prendidas. Como prueba de que el estado es compartido, el plan del PR #35 dio "No changes": el CI leyó del bucket el mismo estado que había dejado un apply local.

Medición: workers locales contra locales más nube

La línea de base corre el Sobel del Hit #1 en una PC con un Ryzen 5 3600 (6 núcleos, 12 hilos) y 31 GiB, con master y workers en Docker Compose (mediana de 3 corridas). Las corridas híbridas usan la misma PC con un worker local más 1 o 2 VMs e2-standard-2 en southamerica-east1-a, creadas y borradas por desbordar.sh en cada corrida. Las cuatro salidas híbridas son idénticas píxel a píxel a la centralizada, y al terminar cada corrida quedaron 0 VMs en el proyecto. Los datos crudos y el log de Terraform de cada corrida (plan y apply) están en hit2/evidencia, junto con la salida del init contra el estado remoto y del plan que corre el CI.

Imagen	1 worker local	2 workers locales	1 local + 1 VM	1 local + 2 VMs
10 MB (1850×1850)	1,31 s	0,86 s	1,75 s	1,54 s
50 MB (4150×4150)	6,12 s	4,55 s	6,40 s	5,26 s

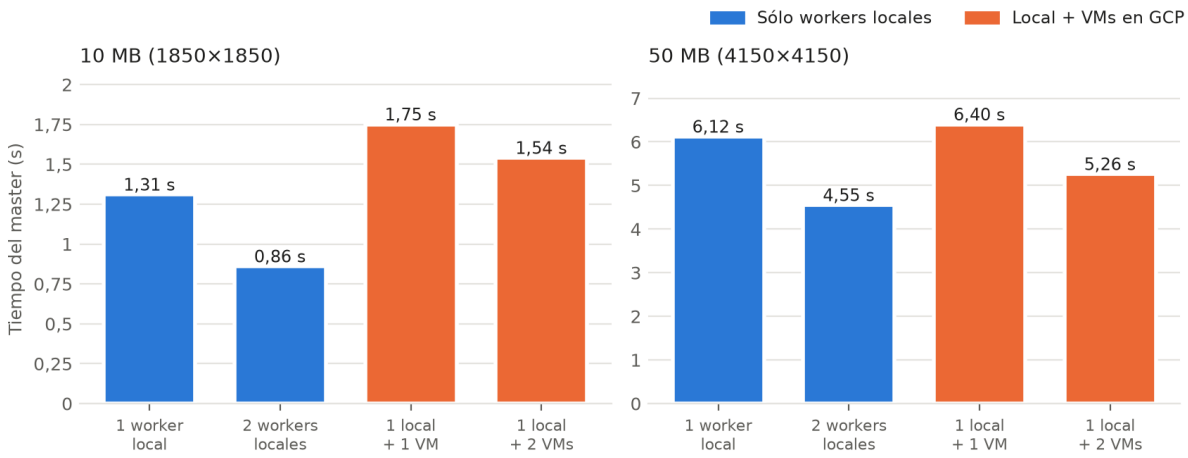


Figura 6.2 — Tiempo del master (lectura, reparto, Sobel y escritura) según dónde están los workers.

Imagen	VMs	Franja local	Franja en la nube	VMs listas	Corrida completa
10 MB	1	0,35 s	1,28 s	100 s	105 s
10 MB	2	0,26 s	1,04 y 1,07 s	140 s	150 s
50 MB	1	1,74 s	3,88 s	108 s	118 s
50 MB	2	1,18 s	2,23 y 2,79 s	93 s	101 s

- **La red pesa más que el cálculo.** Una franja que va a la nube tarda entre 2 y 4 veces más que la misma franja en la PC. El cálculo es el mismo: la diferencia es el viaje de ida y vuelta por Internet.
- **Sólo ayuda con imágenes grandes y poca capacidad local.** Con la imagen de 10 MB la nube nunca ayuda, porque el viaje pesa más que el cálculo. Con la de 50 MB, sumar 2 VMs a un único worker local baja el tiempo de 6,12 a 5,26 s (−14 %), pero dos workers locales siguen siendo más rápidos (4,55 s): mientras haya capacidad local libre, conviene usarla.

- **El arranque no se amortiza con una imagen.** Los tiempos del master no incluyen el arranque: crear las VMs, instalar Docker y bajar la imagen tarda entre 93 y 140 s. Para una sola imagen ese costo no se amortiza nunca.

La conclusión es que el desborde no sirve para que una imagen salga antes, sino para sumar capacidad cuando la local está saturada (muchas imágenes en cola, o imágenes muy grandes frente a pocos workers locales) y mantener las VMs mientras dure esa carga, de modo que el arranque se pague una vez. Es la misma relación entre cómputo y transferencia que señala Armbrust como obstáculo de la nube (sección 5.2).

Dónde corre cada servicio y por qué

Servicio	Dónde	Por qué
Master (parte, reasigna y une)	Local	Tiene la imagen de entrada y escribe la salida: llevarlo a la nube obligaría a subir la imagen entera. Su trabajo es liviano y es quien decide cuándo desbordar.
Workers de base	Local	Atienden la carga normal en equipos que ya están pagados: costo fijo, sin costo por uso ni latencia de red.
Workers de desborde	Nube (VMs efímeras)	Son CPU pura, sin estado e intercambiables. Se crean cuando la carga supera a los locales y se pagan sólo mientras trabajan. Si uno se cae, el master reasigna su franja (Etapa 3 del Hit #1).
Imagen del worker	Registry de la nube	La VM la baja con la identidad de su cuenta de servicio: no quedan claves estáticas en las VMs (R1).
Estado de Terraform	Bucket remoto con lock	Lo comparten los tres integrantes y el CI; el lock evita dos apply a la vez. Nunca va al repositorio.
Pipeline (plan y apply)	GitHub Actions	Se autentica contra la nube por OIDC, con credenciales que duran lo que dura el job.

Las VMs sólo aceptan conexiones al 8080 desde la IP pública de la red local, porque los workers del Hit #1 no tienen autenticación. Las franjas no son datos sensibles, pero viajan por Internet: el tiempo de transferencia entra en la medición, y es el obstáculo de transferencia de datos de la sección 5.2.

7. Hit #3 — Sobel asincrónico en Kubernetes

El Hit #3 es el sistema entero sobre Kubernetes y asincrónico: POST /jobs valida, guarda las imágenes en Redis, encola y responde 202 al instante; el estado se consulta con GET /jobs/{id}. La API, los errores y las interfaces entre servicios están en hit3/contrato.md; el despliegue, en hit3/k8s.

Servicio	Qué hace	Réplicas
backend	API pública: valida, guarda en Redis y encola en sobel.split; JWT con rol usuario o admin	3
split	Parte cada imagen en fragmentos con filas de borde y publica una tarea por fragmento	1
worker	Aplica una operación (grises, sobel, …) a un fragmento; una cola por imagen de worker	N
joiner	Recibe los fragmentos terminados por el fanout, recorta los bordes y une la imagen	2
reasignador	Lee la DLQ y reintenta con límite; lo agotado va a sobel.dlq.agotados	1
frontend	Página para mandar jobs y ver el avance por Server-Sent Events	1+
RabbitMQ, Redis	Colas quorum con DLX; píxeles y estado de cada job	3 y 1

- **Los píxeles van por Redis, no por RabbitMQ:** los mensajes llevan claves y quedan chicos.

- **Una cola por imagen de worker (R2).** Cada paso del pedido elige operación e imagen; dos versiones del worker pueden convivir, y el backend rechaza antes de encolar lo que ningún worker vivo ofrece.
- **Fallas.** Las colas quorum tienen x-delivery-limit: un worker que muere o hace nack manda la tarea a la DLQ. El reasignador la reencola y, si agota los reintentos, la pasa a sobel.dlq.agotados y el job queda en dlq hasta que un admin lo reprocese. Nunca reintenta sin límite.
- **Backend sin estado (R4).** La identidad viaja en cada pedido (JWT HS256 con rol) y el límite por usuario se cuenta en Redis: da lo mismo qué réplica atiende. Un job ajeno da 404, no 403.
- **Registry privado (R1).** Las imágenes son privadas en GHCR; el cluster las baja con un image pull secret y el CI las publica con el GITHUB_TOKEN del job, que vence al terminar.

7.1 Correcciones del 23/10

Corrección pedida	Qué se hizo
El avance del frontend tiene que salir de Redis	El SSE /eventos/{id} lee job:<id> (completos y fragmentos_total) cada 0,5 s y se cierra cuando el job llega a done o failed. El frontend ya no necesita una cola exclusiva por réplica: es sin estado y escala libremente
Un caso que rompa a propósito y muestre reintentos → DLQ → agotados → reprocesar	hit3/evidencia/r_dlq.sh inyecta un fragmento corrupto: el worker hace nack, el reasignador lo reintenta 3 veces, lo pasa a sobel.dlq.agotados, el job queda en dlq, se dispara la alerta SobelDLQAgotadosPresentes y el admin lo reprocesa con /admin/dlq/reprocesar
Logs centralizados	Loki con Fluent Bit en cada nodo (sección 8)
Explicar cómo llegan las métricas a Prometheus	Sección 8 y hit4/README.md §2
Split con partes dinámicas	Si SOBEL_PARTES es 0, el split decide cuántas partes según el tamaño de la imagen, los workers vivos (worker:* en Redis), la cantidad de archivos y los mensajes que ya hay en la cola
Probar con imágenes grandes	Con 100 MP el blur pedía ~1,3 GiB y los workers morían por OOM con 1Gi: el límite pasó a 2Gi (05-workers.yaml)
Alta disponibilidad de RabbitMQ	Cluster de 3 réplicas con colas quorum (consenso Raft) y peer discovery por Kubernetes

7.2 Evidencia en k3s

R1 (hit3/evidencia/r1.sh, 21/10): el mismo pod con y sin el image pull secret.

```
con-secret   1/1     Running            0          15s
sin-secret   0/1     ImagePullBackOff   0          15s
Failed   Failed to pull image "ghcr.io/chimera-sdypp/sobel-backend:dev": ... 401 Unauthorized
```

R4 (hit3/evidencia/r4.sh): backend con 3 réplicas, roles y jobs ajenos.

```
ok   GET /jobs sin token → 401
ok   usuario en GET /stats → 403          ok   admin en GET /stats → 200
ok   ana crea un job → 202                ok   beto pide el job de ana → 404
== Sin estado: 30 consultas al mismo job, las reparte el Service entre las réplicas
```

Prueba de punta a punta (22/10): un job enviado por el ingress terminó en done en 2 s y la imagen descargada fue idéntica a la del Sobel centralizado del Hit #1.

Con la versión 1.1.0 (07/11, en el k3s): RabbitMQ con 3 réplicas forma un solo cluster; la prueba de humo de k6 pasa los 12 chequeos del contrato (401, 422, 202 y un job hasta done); el split con partes dinámicas corta 8 partes para una imagen de 23 MB; y el avance llega al navegador desde Redis (0, 2, 6, 11 y 12 de 12 fragmentos).

Caso de falla provocado (hit3/evidencia/r_dlq.sh; la salida completa está en r_dlq-k3s.txt):

```
job job-corrupto-...: archivo 0, fragmento 0, paso 0 (rejected): intento 2 / intento 3 / intento 4
... agotó los reintentos en el paso 0 (rejected), va a sobel.dlq.agotados
Job pasó correctamente a estado 'dlq' con 3 reintentos y 1 en DLQ.
Mensajes acumulados en sobel.dlq.agotados: 1   → condición de SobelDLQAgotadosPresentes cumplida
POST /admin/dlq/reprocesar → la cola sobel.dlq.agotados quedó limpia (0 mensajes)
Job procesado y finalizado exitosamente (Estado: done).
```

Caída de un nodo de RabbitMQ: con un job de 3 imágenes y 24 fragmentos en curso se borró a la fuerza el pod que era líder de la cola de tareas. Raft eligió otro líder, el job terminó en done en 11 s sin fragmentos en la DLQ, /health siguió en ok y el nodo volvió al cluster.

7.3 En la nube (GKE)

El 07/11 la versión 1.1.0 se desplegó en el GKE compartido con hit3/k8s-gke: un overlay de kustomize que cambia el namespace a chimera-sobel, manda todos los pods a nuestro node pool (con un taint, para que no se mezclen con los del otro grupo) y usa el ingress de GKE. El monitoreo es el del cluster (kube-prometheus-stack y Loki): hit3/k8s-gke/monitoreo.sh agrega nuestros ServiceMonitors, las alertas y el dashboard. Ahí pasan la prueba de humo de k6 (12 de 12 chequeos), la de punta a punta (salida idéntica a la centralizada), el caso de falla de r_dlq.sh (hit3/evidencia/r_dlq-gke.txt) y la caída de un nodo de RabbitMQ con un job en curso.

Un detalle que apareció en la nube: el balanceador de Google rechaza con 411 los POST sin cuerpo que no mandan Content-Length (traefik los aceptaba). El navegador lo manda siempre; r_dlq.sh ahora también.

En GKE el cluster tenía un nodegroup de infraestructura (RabbitMQ, Redis y la observabilidad) y otro de aplicaciones, separados con las etiquetas sobel/infra y sobel/apps; hit3/infra los define, y el plan del CI (hit2/evidencia/plan-ci-hit3-29-10.log) muestra los 17 recursos que crea, entre ellos el grupo de VMs para workers fuera del cluster.

8. Hit #4 — Observabilidad

La observabilidad se instala con el chart kube-prometheus-stack (Prometheus, Alertmanager, Grafana y los exporters del cluster) y Loki con Fluent Bit, todo en el nodegroup de infraestructura.

- **Las métricas se van a buscar (pull).** Cada servicio expone GET /metrics en su puerto; un ServiceMonitor le dice al operador de Prometheus qué Services leer, y Prometheus los consulta cada 10 s. RabbitMQ expone las suyas en :15692. Las de CPU y memoria por pod y por nodo salen de cAdvisor (kubelet) y node-exporter, que instala el chart.
- **Métricas propias.** Tareas procesadas, mensajes en cola, tiempo por tarea, errores por tipo y los tiempos de cada job (definidas en hit3/contrato.md §11).
- **Logs.** Fluent Bit corre en cada nodo (DaemonSet), lee los logs de todos los pods y los empuja a Loki; Grafana los consulta con LogQL al lado de las métricas.
- **Alertas.** hit4/alertrules.yaml define, entre otras, SobelWorkerCaido (no hay workers consumiendo), SobelColaSaturada (la cola supera el umbral) y SobelDLQAgotadosPresentes.

Para probar las alertas se mandaron 10 jobs con 2 workers (terminaron todos), después una ráfaga de 20 jobs y se bajaron los workers a 0: la cola pasó de 32 a 72 mensajes, SobelWorkerCaido y SobelColaSaturada pasaron a pending y después a firing, y Alertmanager las recibió. Al volver los workers, la cola se vació y las alertas se resolvieron.

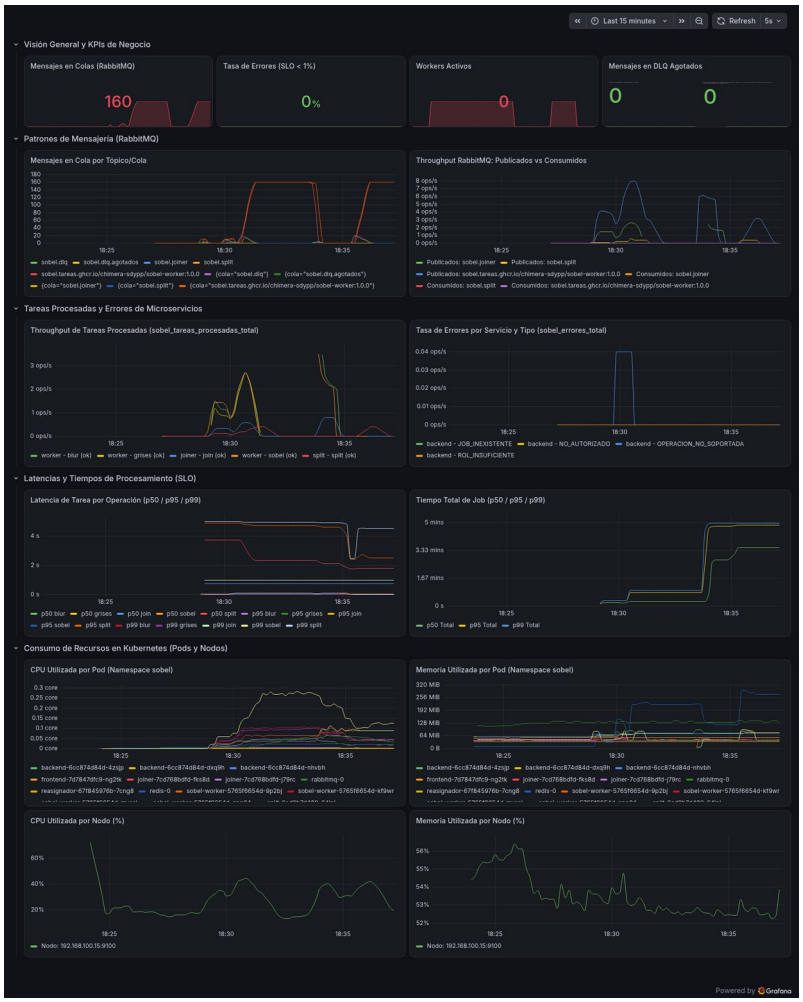


Figura 8.1 — Dashboard de Grafana con la cola creciendo y los workers en 0.

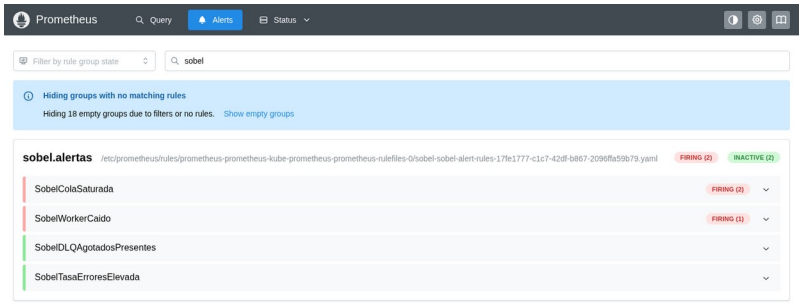


Figura 8.2 — Las alertas en firing en Prometheus.

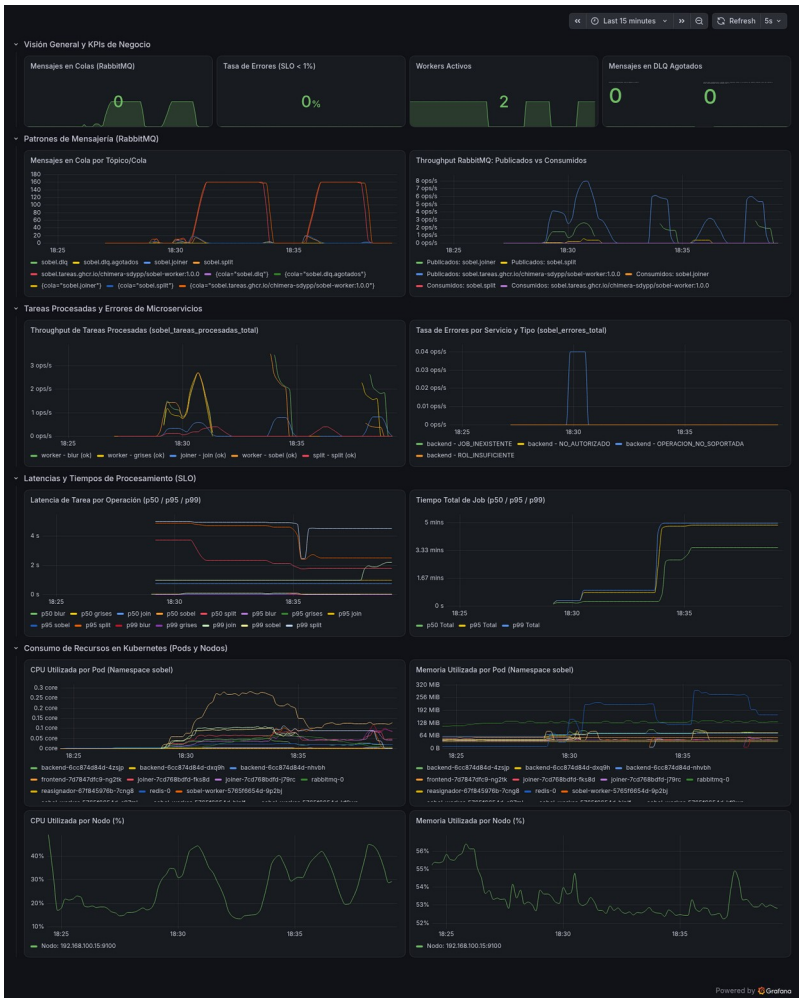


Figura 8.3 — Después de volver a subir los workers: la cola vuelve a 0.

9. Análisis de carga

La carga se mide con k6 (hit3/carga): matriz_r5.js recorre tres variables, V1 tamaño de los datos (de 1 KB a 100 MB), V2 concurrencia (usuarios virtuales) y V3 cantidad de workers; correr_matriz.sh corre cada celda y junta el resumen. Una celda cumple el SLO del contrato (hit3/contrato.md §8) si se cumplen las tres condiciones:

1. p95 de POST /jobs ≤ 1 s;
2. p95 del tiempo total del job ≤ 30 s para pedidos de hasta 10 MB, y ≤ 120 s para los más grandes;
3. menos de 1 % de errores entre los pedidos válidos (5xx, más jobs que terminan failed o dlq sin provocarlo).

Se corrió en el GKE el 07/11, con k6 dentro del cluster (un pod en nuestro node pool) contra la IP pública del ingress: desde una conexión hogareña, subir una imagen de 10 MB ya tarda 2,5–5,7 s y el POST mediría la conexión. Cada usuario virtual usa su propio token. Rampa de 5 s, meseta de 30 s (salvo que se indique otra cosa) y bajada de 5 s; cada usuario manda un job y lo sigue con GET /jobs/{id} hasta que termina, y una parte del tráfico son pedidos inválidos a propósito, que no cuentan como errores. Las métricas de cada fase salen de Prometheus. Datos crudos, tablas y gráfico en hit3/carga/resultados/gke-20261107 (se corrió dos veces; se informa la segunda, que tiene p99, y la primera da los mismos patrones).

V2 — concurrencia (matriz, 2 workers)

VA	VB	VC	V1	Jobs	POST p50 / p95 / p99	Job p50 / p95 / p99	Req/s	Err.	SLO
1	1	1	1 MB	23	108 ms / 124 ms / 131 ms	0,36 / 0,41 / 0,47 s	1,7	0 %	Cumple
1	2	10	10 MB	43	404 ms / 607 ms / 782 ms	16,36 / 19,41 / 19,77 s	8,6	0 %	Cumple
1	2	10	1 MB	107	164 ms / 306 ms / 481 ms	2,08 / 3,05 / 3,41 s	11,0	0 %	Cumple
1	1	50	1 MB	191	200 ms / 478 ms / 615 ms	14,75 / 16,03 / 16,29 s	41,3	0 %	Cumple
10	2	10	1 MB	24	336 ms / 778 ms / 851 ms	25,24 / 28,13 / 29,78 s	7,7	0 %	Cumple
10	2	10	1 MB · cae 1 worker	37	351 ms / 1,1 s / 1,2 s	28,66 / 31,48 / 31,64 s	8,3	0 %	No (POST)
10	3	10	1 MB	22	401 ms / 648 ms / 777 ms	28,74 / 32,19 / 32,21 s	7,7	0 %	Cumple
50	1	10	1 MB	20	1,6 s / 4,4 s / 4,9 s	61,99 / 74,21 / 75,29 s	7,8	98 %	No (POST, errores)

V1 — tamaño (1 archivo, 1 operación, 1 usuario, 2 workers)

V1	Jobs	POST p50 / p95 / p99	Job p50 / p95 / p99	Req/s	Err.	SLO
1 KB	36	124 ms / 186 ms / 193 ms	0,16 / 0,18 / 0,19 s	1,7	0 %	Cumple
1 MB	36	142 ms / 175 ms / 195 ms	0,42 / 0,50 / 0,54 s	1,6	0 %	Cumple
10 MB	14	262 ms / 344 ms / 346 ms	2,65 / 3,06 / 3,24 s	1,2	0 %	Cumple
50 MB	3	764 ms / 834 ms / 840 ms	12,32 / 12,68 / 12,71 s	1,0	0 %	Cumple
100 MB	3	1,8 s / 2,0 s / 2,0 s	25,12 / 26,94 / 27,10 s	0,9	0 %	No (POST)

V3 — cantidad de workers (10 archivos de 1 MB, 2 operaciones, 10 usuarios)

Workers	Jobs	POST p50 / p95 / p99	Job p50 / p95 / p99	Req/s	Err.	SLO
2	25	372 ms / 658 ms / 678 ms	27,25 / 31,93 / 32,44 s	7,6	0 %	Cumple
4	18	488 ms / 879 ms / 896 ms	31,65 / 38,26 / 38,32 s	7,5	0 %	Cumple
8	17	436 ms / 737 ms / 1,0 s	34,97 / 46,67 / 46,77 s	8,1	0 %	Cumple

Req/s cuenta todos los pedidos HTTP, también los GET de seguimiento. Errores: jobs que terminan mal o no terminan sin haberlos provocado.

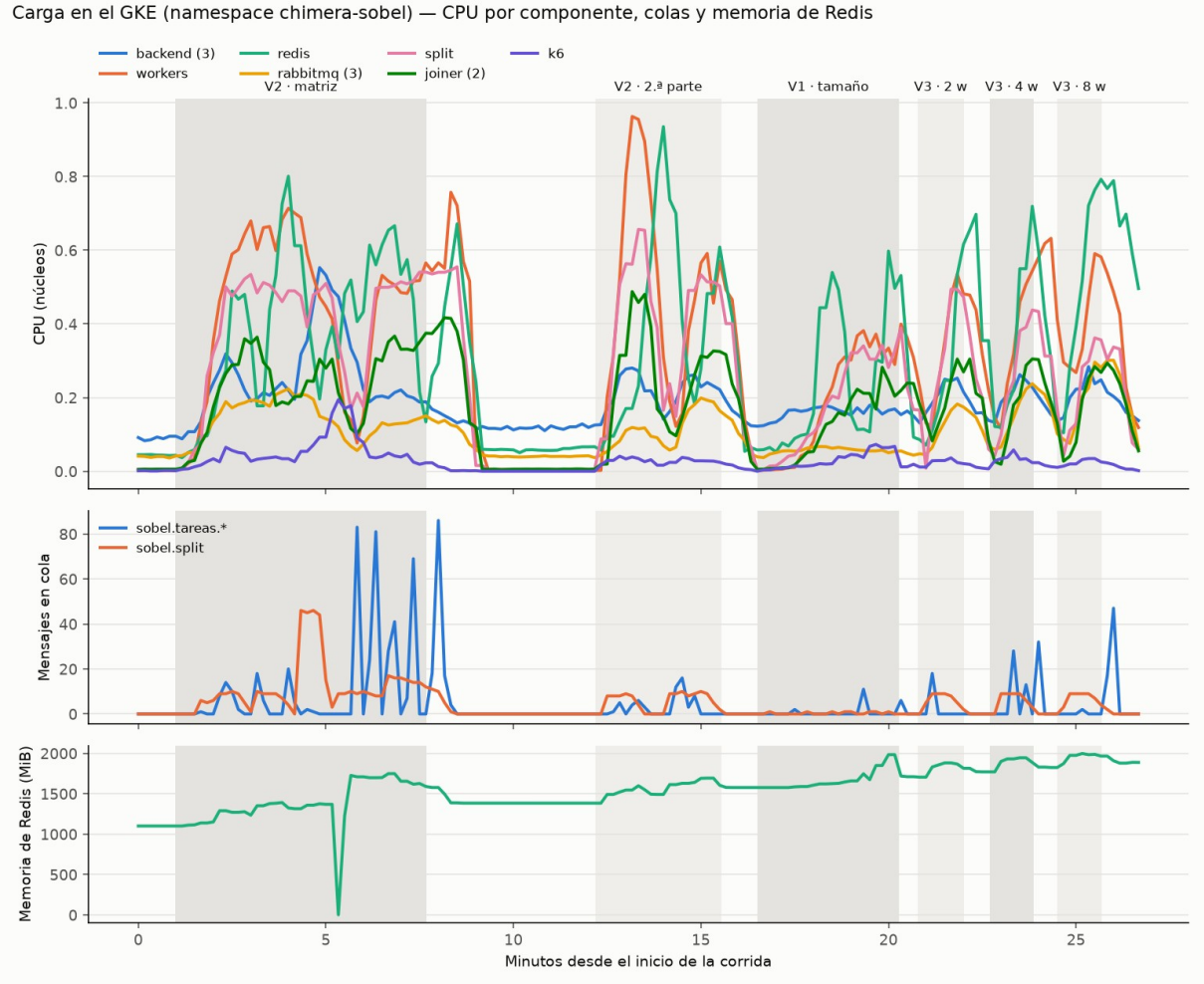


Figura 9.1 — CPU por componente, mensajes en cola y memoria de Redis durante la corrida (las franjas son las fases).

Qué limita al sistema

- **Aceptar el pedido (SLO 1).** Antes de responder 202, el backend lee cada archivo, lo valida y lo guarda entero en Redis. El POST crece con los bytes que entran a la vez: p95 de 175 ms con 1 MB, 834 ms con 50 MB, 2 s con 100 MB y 4,4 s con 50 archivos por pedido. El procesamiento no influye.
- **El split es un paso en serie.** Tiene una réplica y procesa los jobs de a uno: con 50 archivos por pedido su cola llegó a 46 jobs, y ese atraso se arrastra a la celda siguiente (en la primera corrida, la de 10 MB que vino después tardó 60 s; sola, 19 s). Nunca usó más de 0,66 de su núcleo: no le falta CPU, procesa en serie.
- **Más workers no ayudan (V3).** Con 2, 4 y 8 workers el p95 del job subió de 32 a 38 y 47 s. El split corta cada imagen en tantas partes como workers vivos, así que hay más fragmentos que pasan por RabbitMQ y Redis (la CPU de Redis subió de 0,53 a 0,79), mientras el paso que limita, el split, sigue siendo uno.
- **Redis guarda los píxeles y se queda sin memoria.** Con 50 archivos × 10 usuarios, Redis llegó a su límite de 2 GiB y Kubernetes lo mató (OOMKilled): mientras volvía, el backend respondió 503 y los jobs en curso se perdieron (98 % de errores en esa celda). En la primera corrida la misma celda llegó a 1,2 GiB y pasó.

Ningún componente se quedó sin CPU: el mayor pico fue 0,96 núcleos (los workers juntos) sobre 6 disponibles. Lo que cambiaríamos: varias réplicas del split; partes según el tamaño de la imagen y no según la cantidad de workers; maxmemory en Redis para que rechace con un error en vez de morir, y

un TTL más corto para los píxeles; y, para imágenes grandes, subirlas a un almacenamiento de objetos (GCS) y pasar sólo la referencia.

10. Pruebas, CI/CD y seguridad

Componente	Funciones de test	Qué cubren
Hit #0 (4 ejemplos)	30	Productores, consumidores, DLQ y backoff
Hit #1	30	Sobel, franjas y bordes; integración y tolerancia a fallos con Docker
Hit #2	14	Escalador y limpieza del disco del master
Hit #3 · comun	30	Colas, conexión con backoff, claves y métricas
Hit #3 · backend	47	Validación, JWT y roles, límites, errores del contrato; integración con RabbitMQ y Redis
Hit #3 · split	26	Fragmentos con bordes y partes dinámicas
Hit #3 · worker	30	Operaciones, cadena de pasos y fallas
Hit #3 · joiner	21	Unión, recorte de bordes y unión una sola vez
Hit #3 · reasignador	21	Reintentos con límite, agotados y jobs terminados
Hit #3 · frontend	18	SSE desde Redis, autorización, ofertas; integración con RabbitMQ y Redis

El CI (ci.yml) corre en cada push y PR: gitleaks sobre todo el historial (si encuentra un secreto, el pipeline falla), los tests de cada hit, los de integración del Hit #3 contra RabbitMQ y Redis levantados como services, las pruebas con Docker del Hit #1 y la construcción de cada imagen; en cada push a dev o main, y en cada tag, publica las imágenes en GHCR (:dev, :main, :1.0.0 y :sha-<commit>). terraform.yml valida en cada cambio, corre plan en cada PR y apply al mergear a main.

- **Secretos fuera del repo.** .gitignore desde el inicio (.env, *.tfstate, *.tfvars); credenciales por .env.example y Secrets de Kubernetes generados por crear_secretos.sh; gitleaks en el CI.
- **Cero claves estáticas en la nube.** El CI entra a GCP por OIDC con Workload Identity, y el master del Hit #2 en k3s, por Workload Identity Federation con el token de su ServiceAccount. Las VMs usan su cuenta de servicio, que sólo lee el registry.
- **Registry privado.** Image pull secrets en el cluster; las credenciales nunca viajan en un payload ni quedan en una imagen.

11. Herramientas de IA

Integrante	Herramienta	En qué lo ayudó
Salvador Baez	Gemini (Google)	Como asistente de programación en pares: los manifiestos de Kubernetes, conflictos de etiquetas de nodos en k3s, expresiones y agregaciones de PromQL para Grafana y la estructura de la suite de k6 (docs/informe/s6_borg_omega_kubernetes.md, §7). En las pruebas de carga, Google Antigravity (Gemini) para ajustes al runner, consultas PromQL y capturas de Grafana.
Mateo Nomico	Claude Code (Anthropic)	En sus tareas M1 a M8: una spec discutida antes de cada servicio (de ahí salió el contrato del Hit #3), tests escritos antes que el código, revisión de cada tarea contra las reglas del grupo y contra la spec, y la redacción del documento de CAP (docs/informe/m8_cap_y_mensajeria.md, §7).
Tomás Resnik	Claude Code (Anthropic)	Como asistente en la terminal: revisar los PR contra el enunciado y el contrato, encontrar errores, escribir y correr las pruebas de integración en k3s y redactar este informe.

Todo lo generado se revisó, se probó en local y pasó por el CI antes de mergearse. Lo que depende del comportamiento real de RabbitMQ (el límite de entregas, el nack con reencolado, el header x-first-death-queue) se verificó contra RabbitMQ 4 y no se tomó de la IA. La arquitectura, el reparto de tareas y las decisiones de diseño son del grupo.

12. Conclusiones

- **La nube conviene cuando sobra trabajo, no para acelerar uno.** En el Hit #2, una franja en la nube tarda 2 a 4 veces más que en la PC y crear una VM lleva 1 a 2 minutos: desbordar una imagen sola nunca convino. Como servicio que escala por la cola, en cambio, las VMs se crearon sólo cuando el ritmo superó lo que atendían los locales, mantuvieron la cola estable y se borraron al bajar la carga.
- **Lo asincrónico separa aceptar de procesar.** En el Hit #3 el backend responde 202 en milisegundos aunque haya cola; la capacidad se ajusta con workers, y la cola absorbe las ráfagas.
- **CAP en la práctica.** RabbitMQ con colas quorum no pierde tareas aceptadas; Redis, con una sola instancia, da operaciones atómicas para que nada se una dos veces; y como cada paso es idempotente, que RabbitMQ entregue dos veces no rompe nada.
- **Sin observabilidad no se sabe qué falla.** Las métricas de la cola y las alertas mostraron el problema (workers en 0, cola creciendo) antes de que lo notara un usuario, y los logs en Loki permiten seguir un job entre servicios.
- **El límite está en la entrada y en los pasos únicos, no en el cómputo.** En la nube, el procesamiento de los workers nunca fue el problema y ningún componente se quedó sin CPU. Lo que no cumple es aceptar pedidos grandes (el backend guarda las imágenes enteras en Redis antes del 202), el split, que con una sola réplica procesa los jobs de a uno, y la memoria de Redis, que con 50 archivos por pedido llegó a su límite y se reinició. Por eso sumar workers no mejora el tiempo: lo empeora, porque hay más fragmentos. Escalar el split, acotar la memoria de Redis y sacar los píxeles de Redis son los próximos pasos.

13. Referencias

[ARMBRUST10] Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I. y Zaharia, M. (2010). A View of Cloud Computing. Communications of the ACM, 53(4), 50–58.

[BREWER00] Brewer, E. (2000). Towards Robust Distributed Systems. Keynote en PODC 2000.

[BREWER12] Brewer, E. (2012). CAP Twelve Years Later: How the "Rules" Have Changed. Computer, 45(2), 23–29.

[BURNS16] Burns, B., Grant, B., Oppenheimer, D., Brewer, E. y Wilkes, J. (2016). Borg, Omega, and Kubernetes. Communications of the ACM, 59(5), 50–57.

[CBURST] Atlassian. Cloud bursting. atlassian.com.

[GILBERT02] Gilbert, S. y Lynch, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, 33(2), 51–59.

[KUM10] Kumar, K. y Lu, Y.-H. (2010). Cloud Computing for Mobile Users: Can Offloading Computation Save Energy? Computer, 43(4), 51–56.

[NIST800-145] Mell, P. y Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145.

[REDIS-CLUSTER] Redis. Redis cluster specification. redis.io/docs.

[REDIS-REPL] Redis. Redis replication. redis.io/docs.

[RMQ-QQ] RabbitMQ. Quorum Queues. rabbitmq.com/docs/quorum-queues.